Personal Expense Tracker — Full Stack Take-Home Project

A small but complete full-stack application for evaluating a Software Engineering Intern candidate's skills in **React.js**, **Spring Boot (Java 21)**, and **PostgreSQL** using **`JdbcTemplate` with raw SQL** (no JPA / no ORM).

---

1. Overview

Build a simple **Personal Expense Tracker** where an authenticated user can:

- Register and log in
- Create their own spending **categories** (Food, Transport, etc.)
- Record **expenses** under those categories
- Filter and paginate through their expense history
- See a **monthly summary** with totals per category

The scope is intentionally small — what matters is **how it is built**, not how much is built.

---

2. Tech Stack (Mandatory)

Layer	Required
Frontend	**React 18+** (Vite recommended), JavaScript or TypeScript
Styling	Anything (Tailwind / MUI / plain CSS) — minimal polish is fine
Backend	**Spring Boot 3.x** running on **Java 21**
Data Access	**`JdbcTemplate`** or **`NamedParameterJdbcTemplate`** with **raw SQL**
Database	**PostgreSQL 14+**
Auth	**JWT** (stateless)
Build	Maven or Gradle (backend), npm / pnpm / yarn (frontend)

Hard Constraints

- ❌ **No Spring Data JPA**
- ❌ **No Hibernate / EclipseLink / any JPA provider**
- ❌ **No MyBatis / jOOQ / other ORM**
- ❌ **No `BeanPropertyRowMapper`** — write explicit `RowMapper`s
- ❌ **No string concatenation in SQL** — use parameter binding
- ✅ Use **Flyway** or **Liquibase** for schema migrations
- ✅ Use **BCrypt** for password hashing

---

3. Suggested Time

8–12 hours spread over 3–5 days. Quality over completeness — submit what you finish, polished.

---

4. Functional Requirements

4.1 Authentication

- Register with `name`, `email`, `password`
- Login returns a JWT (access token)
- All `/api/**` endpoints except `/api/auth/**` require a valid JWT
- A user can only see and modify **their own** categories and expenses

4.2 Categories

- Per-user. Fields: `name`, `color` (hex `#RRGGBB`), `monthlyBudget` (optional, decimal)
- Full CRUD
- Cannot delete a category that has expenses (return a clean 4xx error) **OR** soft-delete and orphan the expenses to "Uncategorized" — your call, but justify it

4.3 Expenses

- Fields: `amount` (decimal, ≥ 0), `description`, `expenseDate` (date), `categoryId`
- Full CRUD
- List endpoint must support:
  - Date range filter (`from`, `to`)
  - Category filter (`categoryId`)
  - **Pagination** (`page`, `size`)
  - **Sorting** by `expenseDate` or `amount`, asc/desc

4.4 Monthly Summary

A single endpoint returns, for a given `YYYY-MM`:

- Total spent that month
- Per-category breakdown: `{ categoryName, color, total, budget, percentOfBudget }`
- Top 5 most expensive expenses that month

This summary endpoint **must be implemented as a single SQL query using a JOIN + GROUP BY** — not by looping in Java.

Soft delete is mandatory — every read query must filter `WHERE deleted_at IS NULL`. No `DELETE FROM` on these tables.

---

6. REST API

All paths are prefixed `/api`. All non-auth endpoints require `Authorization: Bearer <jwt>`.

Auth

Method	Path	Body	Returns
POST	`/auth/register`	`{ name, email, password }`	`{ id, name, email }`
POST	`/auth/login`	`{ email, password }`	`{ token, user }`

Categories

Method	Path	Notes
GET	`/categories`	List current user's categories
POST	`/categories`	Create
PUT	`/categories/{id}`	Update
DELETE	`/categories/{id}`	Soft delete

Expenses

Method	Path	Notes
GET	`/expenses?from=&to=&categoryId=&page=&size=&sort=`	Paged list
GET	`/expenses/{id}`	One
POST	`/expenses`	Create
PUT	`/expenses/{id}`	Update
DELETE	`/expenses/{id}`	Soft

Method	Path	Notes
delete		

Summary

Method	Path	Notes
GET	`/summary?month=YYYY-MM`	Monthly aggregate

Error Format (consistent)

```
{
  "timestamp": "2026-05-07T10:15:30Z",
  "status": 400,
  "error": "VALIDATION_FAILED",
  "message": "amount must be >= 0",
  "path": "/api/expenses"
}
```

---

7. Frontend Screens

Keep visuals simple. We are not evaluating design taste.

1. **Login / Register** — two simple forms with validation
2. **Dashboard** — summary cards (total this month, # of expenses), per-category breakdown (table or basic progress bars), recent 5 expenses
3. **Expenses** — filter bar (date range + category), paginated table, add/edit modal, delete confirm
4. **Categories** — list with inline create / edit / delete

Required behaviors:

- Auth state persisted across reloads (localStorage is fine — discuss tradeoffs in the README)
- JWT attached via an axios/fetch interceptor
- Protected routes (redirect to `/login` if no token)
- Loading and error states on every async call
- Logout button that clears the token

---

8. Technical Requirements

8.1 Backend

- **Java 21** — use records, switch expressions, pattern matching where they improve clarity
- **Layering**: Controller → Service (interface + impl) → Repository (interface + impl)
  - Naming follows the Facade pattern: e.g. `ExpenseRepositoryJdbcImpl`
- **DTOs** as Java `record` s, separated into `dto.request` and `dto.response` packages
- **Validation**: `@Valid` + `jakarta.validation` annotations on request DTOs
- **Global exception handler** using `@RestControllerAdvice` with the error format above
- **Spring Security** with a stateless JWT filter (no sessions)
- **`@Transactional`** on multi-statement service methods (e.g. delete category + reassign expenses)
- **Use `KeyHolder`** to retrieve the generated `id` after `INSERT`
- **Use `NamedParameterJdbcTemplate`** for queries with multiple bind parameters
- **Connection pool**: HikariCP (Spring Boot default) with sensible `maximum-pool-size`
- **Migrations**: every schema change is a Flyway/Liquibase script, never a manual change
- **Configuration**: secrets read from env vars or `application-*.yml` profiles — never hardcoded

8.2 Frontend

- React functional components + hooks
- React Router for routing
- Auth state via Context (or zustand if preferred — Redux is overkill here)
- Axios with an interceptor that attaches the JWT and handles 401 → logout
- Controlled forms with client-side validation
- No console errors or warnings during normal use

8.3 Security

- BCrypt password hashing (cost ≥ 10)
- JWT signed with a secret loaded from env (`JWT_SECRET`)
- Token expiry ≤ 24h

- CORS configured to allow only the frontend origin
- All SQL is parameter-bound — graders will try basic SQL injection
- Response DTOs **never** include `passwordHash` or any other internal field

8.4 Code Quality

- Clear, consistent package structure
- Sensible commit history (don't squash everything into one commit)
- A `README.md` with: stack, run instructions, env vars, design decisions, what's missing
- A `seed.sql` with one user (with hashed password) + 4-5 categories + 20-30 expenses

---

9. What We Specifically Want To See

These are the *concrete* artefacts that prove each skill:

Skill	Where we'll look
Raw SQL with JOIN + GROUP BY	`/summary` endpoint implementation
`NamedParameterJdbcTemplate`	Expense list query with optional filters
`KeyHolder`	Insert methods that return the new `id`
Custom `RowMapper`	Repository implementations
Transaction handling	A service method that touches multiple tables
Soft delete	Every read query filters `deleted_at IS NULL`
Java 21 features	DTOs as records, `switch` patterns where natural
Spring Security + JWT	Filter chain, no `permitAll()` shortcuts
Validation + error handling	A bad request returns a clean 400 with field errors
React state mgmt	No prop drilling for auth; sensible component split
Async UX	Spinners, disabled buttons during submit, error toasts/messages

---

10. Evaluation Rubric (100 pts)

Area	Points
SQL correctness (queries return right results, JOIN/GROUP BY used appropriately)	15
`JdbcTemplate` usage (RowMappers, KeyHolder, NamedParam, transactions)	15
Spring Boot architecture (layering, DI, DTOs, validation, error handling)	15
Spring Security + JWT correctness	10
Java 21 / general Java quality (records, naming, no over-engineering)	10
React app — works, structure, hooks usage, async UX	15
Database design (constraints, indexes, soft delete, migrations)	10
Security hygiene (BCrypt, parameter binding, no leaked fields, CORS)	5
README, commits, run-ability out of the box	5

11. Bonus (Pick Any — Not Required)

- `expenses_audit` table populated by a Postgres trigger on every insert/update/delete
- Refresh-token flow alongside the access token
- Docker Compose: `postgres` + `backend` + `frontend` come up with one command
- Unit tests: `@JdbcTest` for a repository, `@WebMvcTest` for a controller
- OpenAPI / Swagger UI
- CSV export of expenses
- Simple chart (Recharts / Chart.js) on the dashboard

Mention bonus items in the README so we know to look.

---

12. Submission

1. Push to a **public GitHub repo** with this layout:

```
/backend
/frontend
/db          (migrations, seed.sql)
README.md
```

2. The README must contain **exact** run instructions — graders run a fresh clone.
3. Provide a short **screen recording (≤ 5 minutes)** walking through:
   - The running app
   - The summary SQL query
   - One repository class
   - The JWT filter

Send the repo link and the recording link to: shehan.reshin@tecciance.com

---

13. Things That Will Cost You Marks

- Using JPA / Hibernate / any ORM (instant disqualification of the data layer)
- SQL inside controllers
- Plain-text or weakly hashed passwords
- Hardcoded JWT secrets in code or `application.yml`
- Returning entity objects with `passwordHash` in responses
- `BeanPropertyRowMapper` (we want to see explicit mapping)
- Hard deletes on any base table
- Silent exception swallowing (`catch (Exception e) { }`)
- Building "future-proofing" we didn't ask for (microservices, Kafka, event sourcing…) — keep it scoped
- A README that doesn't let us run the app on the first try

---

14. Questions?

Send any clarification questions before you start — assumptions you make in silence will be evaluated as your design choices.

Good luck!